



s2n-quic v1.55.0

A Rust implementation of the IETF QUIC protocol

Follow

Readme

65 Versions

Dependencies

Dependents

s2n-quic

s2n-quic is a Rust implementation of the [IETF QUIC protocol](#), featuring:

- a simple, easy-to-use API. See [an example](#) of an s2n-quic echo server built with just a few API calls
- high configurability using [providers](#) for granular control of functionality
- extensive automated testing, including fuzz testing, integration testing, unit testing, snapshot testing, efficiency testing, performance benchmarking, interoperability testing and [more](#)
- integration with [s2n-tls](#), AWS's simple, small, fast and secure TLS implementation, as well as [rustls](#)
- thorough [compliance coverage tracking](#) of normative language in relevant standards
- and much more, including [CUBIC congestion controller](#) support, [packet pacing](#), [Generic Segmentation Offload](#) support, [Path MTU discovery](#), and unique [connection identifiers](#) detached from the address

See the [API documentation](#), [examples](#), and [s2n-quic Guide](#) to get started with s2n-quic .

crates.io v1.55.0 docs passing license apache ci failing dependencies 13 out of date MSRV 1.71.0

Installation

s2n-quic is available on [crates.io](#) and can be added to a project like so:

[\[dependencies\]](#)

```
s2n-quic = "1"
```

NOTE: On unix-like systems, [s2n-tls](#) will be used as the default TLS provider. On linux systems, [aws-lc-rs](#) will be used for cryptographic operations. A C compiler and CMake may be required on these systems for installation.

Example

The following implements a basic echo server and client. The client connects to the server and pipes its `stdin` on a stream. The server listens for new streams and pipes any data it receives back to the client. The client will then pipe all stream data to `stdout` .

Server

```
// src/bin/server.rs
use s2n_quic::Server;
use std::{error::Error, path::Path};

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let mut server = Server::builder()
        .with_tls((Path::new("cert.pem"), Path::new("key.pem")))?
```

```

        .with_io("127.0.0.1:4433")?
        .start()?;

while let Some(mut connection) = server.accept().await {
    // spawn a new task for the connection
    tokio::spawn(async move {
        while let Ok(Some(stream)) = connection.accept_bidirectional_stream().await {
            // spawn a new task for the stream
            tokio::spawn(async move {
                // echo any data back to the stream
                while let Ok(Some(data)) = stream.receive().await {
                    stream.send(data).await.expect("stream should be open");
                }
            });
        }
    });
}

Ok(())
}

```

Client

```

// src/bin/client.rs
use s2n_quic::{client::Connect, Client};
use std::{error::Error, path::Path, net::SocketAddr};

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let client = Client::builder()
        .with_tls(Path::new("cert.pem"))?
        .with_io("0.0.0.0:0")?
        .start()?;

    let addr: SocketAddr = "127.0.0.1:4433".parse()?;
    let connect = Connect::new(addr).with_server_name("localhost");
    let mut connection = client.connect(connect).await?;

    // ensure the connection doesn't time out with inactivity
    connection.keep_alive(true)?;

    // open a new stream and split the receiving and sending sides
    let stream = connection.open_bidirectional_stream().await?;
    let (mut receive_stream, mut send_stream) = stream.split();

    // spawn a task that copies responses from the server to stdout
    tokio::spawn(async move {
        let mut stdout = tokio::io::stdout();
        let _ = tokio::io::copy(&mut receive_stream, &mut stdout).await;
    });

    // copy data from stdin and send it to the server
    let mut stdin = tokio::io::stdin();
    tokio::io::copy(&mut stdin, &mut send_stream).await?;

    Ok(())
}

```

Minimum Supported Rust Version (MSRV)

s2n-quic will maintain a rolling MSRV (minimum supported rust version) policy of at least 6 months. The current s2n-quic version is not guaranteed to build on Rust versions earlier than the MSRV.

The current MSRV is [1.71.0](#).

Security issue notifications

If you discover a potential security issue in s2n-quic we ask that you notify AWS Security via our [vulnerability reporting page](#). Please do **not** create a public github issue.

If you package or distribute s2n-quic, or use s2n-quic as part of a large multi-user service, you may be eligible for pre-notification of future s2n-quic releases. Please contact s2n-pre-notification@amazon.com.

License

This project is licensed under the [Apache-2.0 License](#).

Metadata

 13 days ago

 v1.71.0

 Apache-2.0

 92.1 KiB

Install

Run the following Cargo command in your project directory:

```
cargo add s2n-quic
```

Or add the following line to your Cargo.toml:

```
s2n-quic = "1.55.0"
```

Documentation

 docs.rs/s2n-quic/1.55.0

Repository

 github.com/aws/s2n-quic

Owners

 [aws/s2n-core](#)

 [Cameron Bytheway](#)

Report crate

Stats Overview

 **188,168**

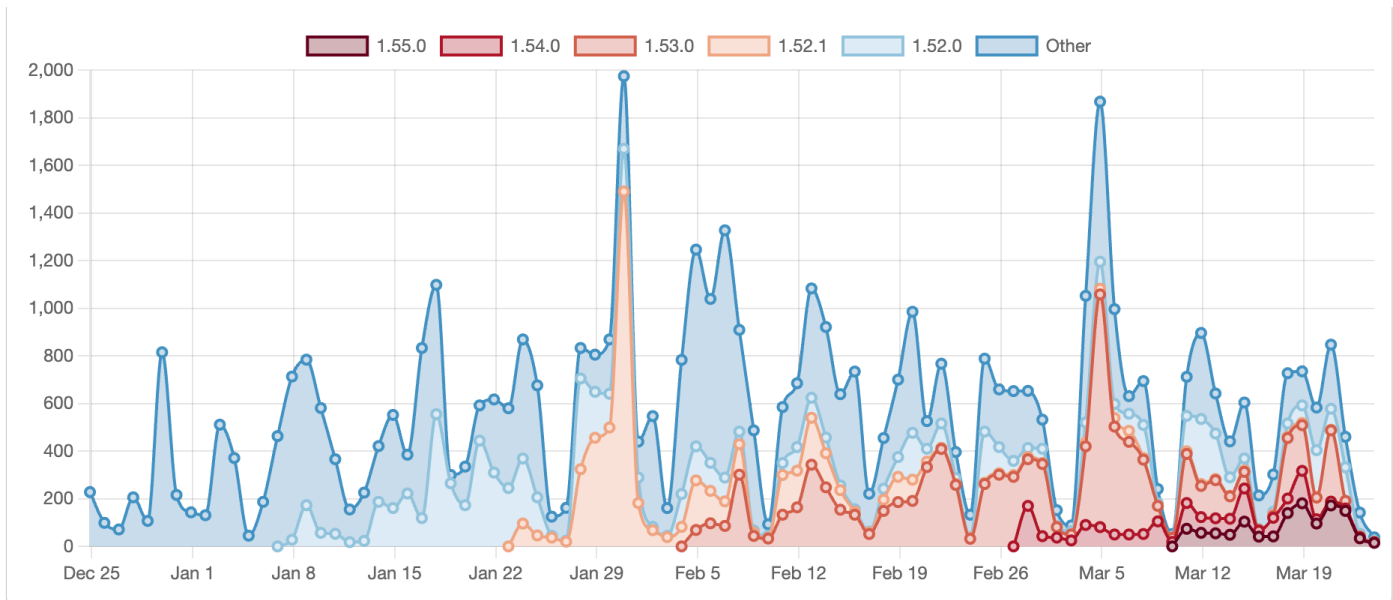
Downloads all time

 **65**

Versions published

Downloads over the last 90 days

Display as **Stacked** ▼



Rust

rust-lang.org

[Rust Foundation](#)

[The crates.io team](#)

Get Help

[The Cargo Book](#)

[Support](#)

[System Status](#)

[Report a bug](#)

Policies

[Usage Policy](#)

[Security](#)

[Privacy Policy](#)

[Code of Conduct](#)

[Data Access](#)

Social

[rust-lang/crates.io](https://github.com/rust-lang/crates.io)

[#t-crates-io](#)

[@cratesiostatus](#)